

TRABALHO PRÁTICO – JARVIS ACADÊMICO (ASSISTENTE INTELIGENTE PARA ESTUDANTES)

1. ORGANIZAÇÃO DO TRABALHO

O projeto será dividido em duas entregas:

Trabalho 1:

- Funcionalidades 3.1, 3.2 e 3.3

Trabalho 2:

- Funcionalidade 3.4
- Melhorias de aprendizado
- Avaliação e análise de erros

Formato:

- Trabalho em duplas
-

2. OBJETIVO

Desenvolver um assistente pessoal acadêmico (JARVIS) capaz de ajudar estudantes a organizar e melhorar seu desempenho utilizando técnicas modernas de Inteligência Artificial.

O sistema deverá integrar:

- RAG (Retrieval-Augmented Generation)
- Tool Calling (chamada de ferramentas)
- Um modelo de linguagem (LLM)

A LLM obrigatória será o Gemma 12B, acessado por meio do token fornecido pelo professor.

O objetivo não é apenas construir um sistema funcional, mas desenvolver um sistema que:

- apoie o aprendizado do usuário
 - integre múltiplas fontes de informação
 - permita avaliação crítica do seu comportamento
-

3. FUNCIONALIDADES OBRIGATÓRIAS

3.1 Consulta a materiais de estudo (RAG)

O usuário deve conseguir fazer perguntas sobre materiais (PDFs, textos, anotações), por exemplo:

- “Explique regressão logística”
- “Resuma o conteúdo sobre embeddings”
- “Quais são os principais pontos do material X?”

O sistema deve:

- carregar documentos
 - dividir em chunks
 - gerar embeddings
 - recuperar trechos relevantes
 - gerar respostas baseadas nesses trechos
-

3.2 Agenda acadêmica

O sistema deve permitir consultas como:

- “O que tenho hoje?”
- “Quais são minhas aulas esta semana?”
- “Tenho prova amanhã?”

A agenda pode ser armazenada localmente (JSON, CSV ou SQLite).

3.3 Lista de tarefas

O sistema deve permitir:

- adicionar tarefa
 - listar tarefas
 - marcar tarefa como concluída
-

3.4 Planejamento de estudos

O sistema deve combinar:

- agenda

- tarefas
- materiais

Exemplos:

- “Monte um plano de estudos para a prova”
 - “O que devo priorizar hoje?”
-

4. TOOL CALLING (OBRIGATÓRIO)

O sistema deve implementar pelo menos 5 ferramentas, por exemplo:

- consultar_agenda
- listar_tarefas
- adicionar_tarefa
- concluir_tarefa
- buscar_material_rag

Requisitos:

- A decisão de chamada deve ser feita pela LLM (não apenas lógica fixa)
 - O sistema deve registrar logs com:
 - ferramenta chamada
 - entrada
 - saída
-

5. MELHORIAS DE APRENDIZADO (OBRIGATÓRIO)

O sistema deve implementar pelo menos 2 funcionalidades voltadas ao aprendizado.

Exemplos:

- geração de exercícios
- perguntas ao usuário (active recall)
- identificação de dificuldades
- recomendação de revisão

Requisito mínimo:

- pelo menos uma funcionalidade deve ser interativa (o sistema pergunta e avalia)
-

6. AVALIAÇÃO DO SISTEMA (OBRIGATÓRIO)

O grupo deve avaliar o sistema com pelo menos 10 perguntas.

Para cada pergunta:

- pergunta
- documentos recuperados
- resposta
- classificação:
 - correta
 - parcialmente correta
 - incorreta

7. ANÁLISE DE ERROS (OBRIGATÓRIO)

Identificar pelo menos 3 falhas.

Para cada falha:

- tipo (recuperação, geração, ambiguidade, etc.)
- causa
- possível solução

8. DATASET (OBRIGATÓRIO)

Cada grupo deve construir seu próprio dataset.

Requisitos:

- mínimo de 10 documentos
- conteúdo acadêmico
- qualidade suficiente para perguntas

Deve incluir:

- origem dos dados
- tipo de conteúdo
- limitações

Entrega:

- pasta /data no repositório ou link externo

Explicar:

- estratégia de chunking
 - impacto no RAG
-

9. QUALIDADE DE ENGENHARIA DE SOFTWARE

O projeto deve demonstrar:

- organização do código
- separação de responsabilidades
- testes básicos
- tratamento de erros
- logs

Uso de IA para:

- revisão
- sugestão de melhorias
- identificação de bugs

O aluno deve conseguir explicar o código.

10. TECNOLOGIAS E RESTRIÇÕES

O trabalho deve ser desenvolvido de modo que o grupo tenha controle sobre o sistema implementado.

Diretrizes:

- ferramentas gratuitas são permitidas
- o grupo deve implementar explicitamente:
 - RAG
 - integração com LLM
 - tool calling

LLM obrigatória:

- Gemma 12B

Regras:

- não utilizar ferramentas que gerem o sistema completo automaticamente (ver seção 13)
- ferramentas de apoio ao desenvolvimento são permitidas

O acesso a LLM pode ser feito via código abaixo

None

```
• from openai import OpenAI
•
• client =
  OpenAI(base_url='https://llm.liaufms.org/v1/gemma-3-12b-i
    t',
    api_key='Cxt2ftLF7d3mHS2JdiFqB-eSDAQeZvFATPXP021V9A')
• resp = client.chat.completions.create(
•     model='google/gemma-3-12b-it',
•     messages=[{'role': 'user', 'content': 'Hi'}],
• )

print(resp.choices[0].message.content)
```

11. USO DE IA PARA DESENVOLVIMENTO

Permitido e incentivado.

Requisitos:

- entender o código
- listar ferramentas usadas no README
- estar preparado para explicar o sistema

12. ENTREGA

12.1 Código

- repositório GitHub
- README com instruções
- lista de IAs utilizadas

12.2 Dataset

- mínimo 10 documentos
- pasta /data ou link
- documentação com:
 - origem
 - tipo

- limitações
- chunking

12.3 Vídeo (até 3 minutos)

Deve mostrar:

1. arquitetura
2. sistema funcionando

Não entregar itens obrigatórios = nota zero

13. ITENS PROIBIDOS E PERMITIDOS

Proibidos:

- compartilhar código entre grupos
- usar ferramentas que geram o sistema completo automaticamente

Exemplos:

lovable, Bolt.new, v0, Pythagora AI, Replit Ghostwriter, Flowise, Retool AI, Superblocks AI, BuildShip, Glide, Bubble, Appsmith, ToolJet

Permitidos:

Claude Code, Copilot, Cursor, Codeium, Amazon CodeWhisperer, Continue.dev, Ollama, OpenDevin, Devin e similares

14. CRITÉRIOS DE AVALIAÇÃO

Funcionalidade: 20%

RAG: 20%

Tool calling: 15%

Avaliação + erros: 20%

Aprendizado: 15%

Engenharia: 10%

Cada dia de atraso na entrega, incorre na perda de 1 ponto por dia.

15. DIFERENCIAL

- interface gráfica
- melhor qualidade
- melhorias avançadas

- integração mais inteligenteCaso o professor entenda que houve diferenciais do grupo, ou que o grupo fez algo além do pedido, o grupo poderá receber bônus de até 2 pontos.
-